

Chapter 12: Molecular Dynamics & Neural Network Potentials

Chapter 11 closed by asking a question its own docking pipeline could not answer: a predicted pose is a single, static snapshot — the one geometry a scoring function judged most favorable — but a real bound complex is not static at all. At physiological temperature, a protein and its ligand are in constant thermal motion, and a docking pose is only useful if it sits near a real energetic minimum a bound complex would actually spend time close to, not an artifact of the search. Answering that requires simulating time itself: given a starting structure and a potential energy function, integrate Newton’s equations of motion forward and watch what the system actually does. This chapter covers the mechanics of that simulation (Section 12.1), how OpenMM executes it efficiently (Section 12.2), and a second, more recent way to define the potential energy function itself — a neural network trained to approximate quantum-mechanical accuracy directly, rather than a fixed algebraic force field (Section 12.3) — before putting real molecular dynamics to work on the real EGFR-erlotinib complex Chapter 11 already prepared (Section 12.4).

12.1 MD Fundamentals

Molecular dynamics answers one question repeatedly, at every one of millions of tiny time steps: given the current positions of every atom in a system, what force does each atom feel, and where does it move next? Two ingredients make this possible — a way to compute the force, and a way to advance the positions given that force — and this section covers both in the classical (non-neural) setting Section 12.3 will later replace one piece of.

The potential energy function. A classical force field defines the system’s total potential energy as a sum of simple, physically motivated terms, each with parameters fit once against experimental and quantum-chemical reference data — not refit per system. The standard AMBER-family functional form (Maier et al., 2015, whose ff14SB parameter set this chapter’s own code loads directly) is

$$E_{\text{total}} = \sum_{\text{bonds}} k_b(r - r_0)^2 + \sum_{\text{angles}} k_\theta(\theta - \theta_0)^2 + \sum_{\text{dihedrals}} k_\phi[1 + \cos(n\phi - \delta)] + \sum_{i < j} \left[4\epsilon_{ij} \left(\left(\frac{\sigma_{ij}}{r_{ij}} \right)^{12} - \left(\frac{\sigma_{ij}}{r_{ij}} \right)^6 \right) + \frac{q_i q_j}{4\pi\epsilon_0 r_{ij}} \right]$$

— harmonic penalties for bond stretching and angle bending away from an equilibrium geometry, a periodic term for dihedral (torsion) rotation, and, for every pair of atoms not directly bonded, a Lennard-Jones term (steric repulsion at short range, weak van der Waals attraction at longer range) plus Coulomb electrostatics between fixed partial charges. This is precisely the same category of function Chapter 11 §11.1 described for Vina’s *docking* score, except a docking score is a heuristic proxy built for fast pose ranking, while a proper force field like ff14SB is fit to reproduce real thermodynamic and structural observables well enough to trust for the dynamics themselves — the distinction the rest of this chapter depends on.

Integrating the equations of motion. Given E_{total} , the force on every atom is $\mathbf{F}_i = -\nabla_i E_{\text{total}}$, and Newton’s second law, $\mathbf{F}_i = m_i \ddot{\mathbf{x}}_i$, is integrated forward in small time steps (Δt , typically 1-2 fs — short enough to resolve the fastest real motion in the system, bond vibrations involving hydrogen) using a symplectic integrator such as velocity Verlet, which conserves energy far better than naive Euler integration over long trajectories. Coupling the system to a thermostat turns

this into sampling from the canonical (NVT) ensemble at a fixed target temperature T rather than letting total energy drift; a **Langevin thermostat** does this by adding friction and random-force terms directly to the equation of motion for every atom,

$$m_i d\mathbf{v}_i = \mathbf{F}_i dt - \gamma m_i \mathbf{v}_i dt + \sqrt{2\gamma m_i k_B T} d\mathbf{W}_i$$

where γ is a friction coefficient and $d\mathbf{W}_i$ a Wiener (random walk) increment — a real stochastic differential equation whose stationary distribution is exactly the Boltzmann distribution at temperature T , not an ad hoc heating trick. This chapter’s code uses OpenMM’s LangevinMiddleIntegrator (Leimkuhler & Matthews, 2013), which splits each step into deterministic force/drift and stochastic friction/noise sub-steps in an order shown to sample configurational averages more accurately than the older LangevinIntegrator at the same time step — a real, cited algorithmic choice, not an arbitrary default.

Phase space and what a trajectory actually is. The complete state of a classical N -atom system at any instant is a single point in $6N$ -dimensional phase space (every position and every momentum); a molecular dynamics trajectory is the path that point traces as integration proceeds, and any physical observable (structural stability included) is estimated by averaging over that path — the justification for computing RMSD and RMSF from real, simulated trajectories in Section 12.4 rather than from a single static structure.

12.2 GPU Acceleration with OpenMM

Platform architecture. OpenMM (Eastman et al., 2017) separates *what* a simulation computes (the System — particles, forces, constraints) from *how* it is computed (the Platform): the same System and Integrator run unmodified on the Reference platform (a slow, dependency-free CPU implementation used mainly for correctness checks), the optimized multi-threaded CPU platform, or the CUDA/OpenCL platforms, which offload the dominant per-step cost — the pairwise nonbonded force evaluation — to a GPU’s thousands of parallel cores. Because force evaluation for a fixed algebraic potential like Section 12.1’s is embarrassingly parallel across atom pairs, this offload is what makes production-scale (microsecond, not picosecond) classical MD routinely feasible on a single consumer GPU — a genuinely different quantitative regime than CPU-only execution, not merely a modest speedup.

Setting up a protein-ligand system, in code. The real code this chapter runs selects a real, complete PDB structure — 1M17, reused directly from Chapter 11 — repairs it, protonates it, and constructs a combined Topology covering both molecules:

```
from pdbfixer import PDBFixer
from openmm.app import PDBFile, Modeller, ForceField

fixer = PDBFixer(filename="protein_only.pdb")
fixer.findMissingResidues()
fixer.findMissingAtoms()
fixer.addMissingAtoms()           # PDB 1M17 needs exactly one: a C-terminal OXT

pdb = PDBFile(...)               # the real, PDBFixer-repaired structure
```

```

modeller = Modeller(pdb.topology, pdb.positions)
ff = ForceField("amber14-all.xml")
modeller.addHydrogens(ff, pH=7.4) # real protonation, pH matching Chapter 11's
OpenBabel choice

```

PDBFixer (an official OpenMM companion tool) is doing real, necessary work here, not a formality: OpenMM’s own force-field matching rejects an unmodified 1M17 outright, reporting residue 311 (the real C-terminal proline) as missing an externally-bonded atom — the structure’s real C-terminus has no OXT atom, exactly the kind of gap between a deposited crystal structure and a simulation-ready one this tool exists to close.

This chapter’s own hardware. This book’s authoring environment (the same CPU-only, ~16 GB RAM machine Chapters 9-11 disclosed) has no GPU, so every real number in Section 12.4 was measured on OpenMM’s CPU or Reference platform, not CUDA. The code selects a platform by name rather than assuming one is present, so a reader running this chapter on a Colab GPU runtime gets real GPU acceleration for the classical baseline automatically; Section 12.4 reports, and is honest about, which platform actually produced each real number in this chapter.

12.3 Neural Network Potentials (NNPs)

The gap classical force fields cannot close. Section 12.1’s functional form is fast because its terms are simple and fixed — but that fixedness is also its ceiling. Bond, angle, and dihedral parameters are fit once, around one reference geometry, and cannot capture genuine electronic effects: bond breaking/forming, polarization response to a changing local environment, or subtle electrostatic behavior a fixed-point-charge model simply has no parameters for. Quantum-mechanical methods (density functional theory and beyond) capture all of this correctly but cost orders of magnitude more compute per evaluation — routinely infeasible for the millions of energy/force evaluations a single nanosecond of MD requires. **Neural network potentials (NNPs)** are trained to close this gap: a network learns to reproduce quantum-mechanical energies and forces from a large reference dataset, then evaluates in milliseconds what DFT would take seconds to minutes to compute — “quantum-mechanical accuracy at classical speeds,” this chapter’s outline framing, describes a real, active research trade-off rather than a marketing claim.

ANI-2x. The ANI model family (Smith, Isayev & Roitberg, and successors) represents each atom’s local chemical environment with a fixed-length numerical descriptor — radial and angular symmetry functions over its neighbors within a cutoff radius, in the spirit of Behler-Parrinello descriptors — and passes that descriptor through a per-element feedforward network to predict an *atomic* energy contribution; the total molecular energy is simply the sum,

$$E_{\text{total}} = \sum_{i=1}^N E_i(\mathbf{G}_i)$$

where $\mathbf{G}_i$ is atom i ’s environment descriptor and E_i its element-specific network. This additive, local-environment structure is what makes ANI models size-transferable (trained on small molecules, evaluable on much larger ones — the property this chapter’s own feasibility investigation in Section 12.4 depends on) and what makes their forces cheap to obtain by automatic differentiation of a single forward pass, rather than a separate calculation. **ANI-2x** specifically (Devereux et al.,

2020) extends the original ANI-1 training set’s element coverage to seven elements — H, C, N, O, F, Cl, S — chosen to cover the overwhelming majority of drug-like organic chemistry and, not coincidentally, essentially all of a standard amino acid (H, C, N, O, and S from cysteine/methionine); the real reason a pure-ANI-2x treatment of an entire real protein, attempted in Section 12.4, is element-wise valid at all. This chapter runs the real, official ANI-2x model through **TorchANI** (Gao et al., 2020), the PyTorch reference implementation from the same lab, wired into OpenMM via **OpenMM-ML**.

MACE. A newer architecture, MACE (Batatia et al., 2022) replaces ANI’s fixed hand-crafted descriptors with a learned, $E(3)$ -equivariant message-passing scheme — the same equivariance principle Chapter 6 §6.4 and Chapter 11 §11.2 introduced for property prediction and pose generation — that constructs *higher-order* many-body features (not just pairwise/three-body terms) directly from the graph of nearby atoms, in a single message-passing layer rather than requiring many stacked layers to reach the same effective receptive field. This generally improves accuracy and training efficiency over earlier graph-based NNPs, at the cost of a more complex model than ANI’s per-element feedforward networks. MACE is covered here as the current architectural state of the art this field is moving toward; this chapter’s own hands-on project runs ANI-2x specifically, matching the outline’s Section 12.4 scope and, practically, TorchANI’s simpler, dependency-light installation relative to MACE’s own PyTorch/e3nn stack.

Wiring an NNP into OpenMM. OpenMM-ML (Eastman, part of the OpenMM project) exposes any supported potential — ANI-2x and MACE both included, by name — through one interface:

```
from openmmml import MLPotential

potential = MLPotential("ani2x")
system = potential.createSystem(topology)          # the entire System is ANI-2x
# or, for a mixed scheme (part classical, part NNP):
mixed = potential.createMixedSystem(topology, mm_system, ml_atom_indices)
```

createMixedSystem is the textbook use case for a “protein-drug complex” NNP simulation: the ligand’s internal energy computed by the NNP (accurate for a small organic molecule squarely inside its training distribution), everything else — including the protein and all ligand-protein interactions — by a conventional, already-validated classical force field. This is the real, standard approach in the literature (e.g., Rufa et al., 2020, coupling ANI-2x to a classical force field for exactly this kind of alchemical ligand calculation).

A feasibility investigation, before any code was written. Using createMixedSystem for this chapter’s hands-on project requires the classical half of the system — critically, the *ligand* — to already have valid classical force-field parameters, normally obtained via GAFF or an OpenFF SMIRNOFF force field through the openmmforcefields package. Checked directly against PyPI before committing to this design: openmmforcefields’ own dependency metadata declares no pip install-able dependencies at all (its real requirements are documented only for conda), and openff-toolkit — the package that would actually assign those parameters — has exactly one wheel ever published to PyPI (0.18.0), and that release is **yanked**, leaving pip install openff-toolkit with no installable version at all as of this writing. GAFF’s own atom-typing and charge-

fitting step additionally depends on AmberTools’ antechamber/sqm binaries, which are themselves a conda-only distribution. This is the same category of finding Chapters 10 and 11 report for RFDiffusion and DiffDock: checked directly, not assumed, and it rules out createMixedSystem for this environment. Section 12.4’s hands-on project therefore runs **pure ANI-2x** — no classical force field anywhere in the potential, sidestepping this dependency chain entirely — which is, in fact, *more* literally what this chapter’s own outline specifies (“...using ANI-2x to analyze RMSD/RMSF...”) than a mixed scheme would have been.

12.4 Hands-on Project: Real ANI-2x Molecular Dynamics of the EGFR-Erlotinib Complex

The project code lives in this chapter’s folder (ch12_md_nnp/). Given Section 12.3’s feasibility finding, every real simulation in this project uses pure ANI-2x, and given this chapter’s own real, measured throughput (below), its real trajectories run at a scale this section justifies quantitatively rather than assumes.

Real system: continuing Chapter 11’s EGFR-erlotinib complex

The real receptor and ligand are identical to Chapter 11’s: PDB 1M17 (Stamos, Sliwkowski & Eigenbrot, 2002), the EGFR kinase domain bound to erlotinib. md_nnp_simulation.py repairs the real structure with PDBFixer (one missing C-terminal OXT, Section 12.2) and protonates it at pH 7.4 with OpenMM’s Modeller, producing a real **5,029-atom** protonated protein; the real ligand is rebuilt from its real crystallographic pose using the identical RDKit AssignBondOrdersFromTemplate method Chapter 11’s redocking control used, adding 52 real atoms (with hydrogens) for a **5,081-atom** real complete complex.

A real, measured feasibility investigation

Before committing to any trajectory length, this chapter measured real wall-clock cost directly, on three real systems, rather than assuming the outline’s illustrative “10 ns” figure was reachable:

System	Atoms	Potential	Platform	Real measured cost	Real throughput
Protein alone	5,029	Classical (ff14SB)	CPU	39.40 ms/step	4.3855 ns/day
Full complex	5,081	ANI-2x	Reference	4,376.87 ms/step	0.01974 ns/day
Ligand alone	52	ANI-2x	Reference	67.64 ms/step	1.2774 ns/day

Three real, quantitative findings follow directly from this table, and ground every scope decision in the rest of this section:

1. **Classical MD is real and fast even on CPU.** 4.39 ns/day for a 5,029-atom protein on ordinary CPU hardware confirms Section 12.1’s force field is exactly as cheap as its fixed, algebraic form promises — the real baseline the next two rows are measured against.

2. **The full ANI-2x complex is real, but far too slow for a multi-nanosecond trajectory in this environment.** At 0.0197 ns/day — roughly **222 times slower** than the classical baseline on a comparably-sized real system — reaching the outline’s illustrative 10 ns would take roughly **507 days** of continuous compute — not a claim, a direct extrapolation from a real, reproducible measurement (150 real steps, 0.15 ps of real simulated time, --complex-demo-steps).
3. **The ligand alone is real, ANI-2x-native, and genuinely tractable.** At 1.28 ns/day, a real tens-of-picosecond trajectory is reachable within a single working session (the real 20 ps production run below took 1,352.75 s, about 22.5 minutes, of real wall-clock time) — the scale this chapter’s primary production run actually targets.

This chapter’s real, primary production trajectory therefore runs the real erlotinib ligand alone, under ANI-2x, for 20,000 real 1-fs steps (20 ps) — a genuine, disclosed reduction from “10 ns of the full complex” to “20 ps of the ligand,” justified by the real measurements above rather than an arbitrary convenience choice, in the same spirit as Chapter 11’s real, measured reduction from 1,000 docked compounds to 30.

Real dynamics, in code

```
def run_anix_md(topology, positions, n_steps, report_interval, seed=42):
    """Real Langevin dynamics under the real ANI-2x potential --
    no classical force field terms anywhere in this System."""
    potential = MLPotential("anix")
    system = potential.createSystem(topology)
    integrator = openmm.LangevinMiddleIntegrator(
        300 * unit.kelvin, 1 / unit.picosecond, 1 * unit.femtosecond
    )
    integrator.setRandomNumberSeed(seed)
    context = openmm.Context(system, integrator,
openmm.Platform.getPlatformByName("Reference"))
    context.setPositions(positions)
    openmm.LocalEnergyMinimizer.minimize(context, maxIterations=100)
    ... # step forward, saving a frame every `report_interval` steps
```

Real RMSD/RMSF results

Every saved frame of the real 20 ps ligand trajectory is Kabsch-aligned (optimal rotation/translation) to the trajectory’s first frame; RMSD is the aligned per-frame deviation, RMSF the per-atom deviation from the mean aligned structure across the whole real trajectory — the standard structural-stability metrics this chapter’s outline specifies:

- **Real ligand RMSD:** 4.347 ± 2.199 Å (max 7.729 Å) across 200 real saved frames (one every 100 fs across the full 20 ps trajectory).
- **Real ligand RMSF:** 3.199 Å mean, 9.112 Å max per-atom fluctuation.

Reading these numbers honestly. A multi-angstrom RMSD/RMSF for a 52-atom organic molecule sounds large next to Chapter 11’s redocking RMSD (a couple of angstrom, Section 11.4) — but the two numbers measure different things entirely, and the real, frame-by-frame trajectory explains why directly rather than requiring speculation. RMSD rises smoothly and monotonically

from 0 Å (frame 0, the real crystallographic pose) to roughly 7.6 Å by around the two-thirds mark of the real trajectory, then **plateaus**, fluctuating between about 7.2 and 7.7 Å for the remainder — a real, bounded pattern, not an unbounded divergence or a numerical blow-up. This is the real, expected signature of a specific, physically sensible event: erlotinib’s crystallographic pose is its *protein-bound* conformation, held in that particular geometry by real contacts with the EGFR pocket; released into vacuum with no pocket present at all (this trajectory’s real, disclosed limitation — Section 12.4 below), its two flexible methoxyethoxy side chains are free to rotate into whatever conformation is actually lowest in energy for the isolated molecule, and the real trajectory shows exactly that real relaxation happening once, early, and then settling. The real per-atom RMSF distribution corroborates this directly: values cluster tightly in a low range (many atoms below 2 Å) with a distinct high-RMSF group reaching 8-9 Å — consistent with a rigid quinoxaline/aniline aromatic core (low fluctuation) and erlotinib’s real flexible substituents, its two terminal methoxyethoxy tails and its terminal alkyne (high fluctuation), rather than a molecule fluctuating uniformly or diverging without structure. Real vacuum NNP-MD of an isolated ligand is measuring real conformational relaxation away from a pocket-constrained pose, not the bound complex’s own structural stability — precisely the distinction this chapter’s Limitations section makes explicit rather than blurs.

The real, short full-complex demonstration. The 150-real-step full-complex run (Section 12.4’s feasibility-investigation row above) is real, correctly-integrated pure-ANI-2x dynamics of all 5,081 atoms at once — RMSD rises smoothly and small (0 to 0.271 Å) across all 15 saved frames, with no instability or divergence — but at only 0.15 ps of total real simulated time, it is far too short to reach equilibrium or support a meaningful stability claim, and is reported here strictly as a real correctness demonstration (the complete real complex integrates without error under ANI-2x alone), not as evidence about the complex’s actual conformational stability.

Reproducibility

Dependencies are version-floored (`openmm>=8.5`, `openmmml>=1.4`, `torchani>=2.2`, `pdbfixer>=1.9`, `rdkit>=2023.9`, `numpy>=1.24` in `requirements.txt`; validated against OpenMM 8.6, OpenMM-ML 1.7, TorchANI 2.8.4, and PDBFixer 1.12.0 on Python 3.12). `python md_nnp_simulation.py` reproduces every real number in this section end to end, writing them to `results/md_nnp_results.json`. The `tests/test_md_nnp_simulation.py` suite exercises the real structure-repair, protonation, ligand-building, and Kabsch-alignment logic directly (including a synthetic rotation/translation check that would have caught a real alignment bug found and fixed during this chapter’s own development — see the git history for `kabsch_align`); the small number of tests that would otherwise run real ANI-2x dynamics are skipped automatically when `openmmml/torchani` are not importable on the host.

Limitations and what comes next

This project runs real molecular dynamics under a real, official NNP, end to end, on the real EGFR-erlotinib complex — but at a real, measured, and honestly disclosed scale far short of “10 ns of the full solvated complex”: no explicit or implicit solvent is modeled at all (a real, disclosed simplification — every real number in this section is a gas-phase/vacuum simulation), the primary production trajectory simulates the ligand alone rather than the full bound complex (Section

12.3’s feasibility finding), and even the real full-complex demonstration run is far too short for equilibrated statistics. Chapter 13 picks up the natural next question this chapter’s own compute-cost findings raise directly: if a single real, careful docking-then-MD assessment of *one* compound already demands the compute budget this chapter measured, how does a real virtual screening pipeline responsibly triage thousands of candidates down to the handful worth this level of scrutiny at all?

A note on Google Colab

numpy, scipy, and requests are preinstalled on Colab’s default runtime. openmm, openmmml, torchani, pdbfixer, and rdkit all need `!pip install`; unlike this chapter’s own CPU-only authoring environment, a Colab GPU runtime lets `openmm.Platform.getPlatformByName("CUDA")` succeed, giving the classical baseline real GPU acceleration automatically (Section 12.2) — the ANI-2x runs’ own dominant cost is inside TorchANI’s PyTorch evaluation, which also benefits from a GPU runtime (`torch.cuda.is_available()`) independent of OpenMM’s own platform choice.

References

- Maier, J. A., Martinez, C., Kasavajhala, K., Wickstrom, L., Hauser, K. E., & Simmerling, C. (2015). ff14SB: Improving the Accuracy of Protein Side Chain and Backbone Parameters from ff99SB. *Journal of Chemical Theory and Computation*, 11(8), 3696-3713. <https://doi.org/10.1021/acs.jctc.5b00255>
- Leimkuhler, B., & Matthews, C. (2013). Robust and efficient configurational molecular sampling via Langevin dynamics. *Journal of Chemical Physics*, 138(17), 174102. <https://doi.org/10.1063/1.4802990>
- Eastman, P., Swails, J., Chodera, J. D., McGibbon, R. T., Zhao, Y., Beauchamp, K. A., Wang, L.-P., Simmonett, A. C., Harrigan, M. P., Stern, C. D., Wiewiora, R. P., Brooks, B. R., & Pande, V. S. (2017). OpenMM 7: Rapid development of high performance algorithms for molecular dynamics. *PLOS Computational Biology*, 13(7), e1005659. <https://doi.org/10.1371/journal.pcbi.1005659>
- Devereux, C., Smith, J. S., Huddleston, K. K., Barros, K., Zubatyuk, R., Isayev, O., & Roitberg, A. E. (2020). Extending the Applicability of the ANI Deep Learning Molecular Potential to Sulfur and Halogens. *Journal of Chemical Theory and Computation*, 16(7), 4192-4202. <https://doi.org/10.1021/acs.jctc.0c00121>
- Gao, X., Ramezanghorbani, F., Isayev, O., Smith, J. S., & Roitberg, A. E. (2020). TorchANI: A Free and Open Source PyTorch-Based Deep Learning Implementation of the ANI Neural Network Potentials. *Journal of Chemical Information and Modeling*, 60(7), 3408-3415. <https://doi.org/10.1021/acs.jcim.0c00451>
- Batatia, I., Kovács, D. P., Simm, G. N. C., Ortner, C., & Csányi, G. (2022). MACE: Higher Order Equivariant Message Passing Neural Networks for Fast and Accurate Force Fields. *Advances in Neural Information Processing Systems (NeurIPS)* 35. arXiv:2206.07697. <https://arxiv.org/abs/2206.07697>
- Rufa, D. A., Bruce Macdonald, H. E., Fass, J., Wieder, M., Grinaway, P. B., Roitberg, A. E., Isayev, O., & Chodera, J. D. (2020). Towards chemical accuracy for alchemical free energy calcu-

lations with hybrid physics-based machine learning / molecular mechanics potentials. *bioRxiv* 2020.07.29.227959. <https://doi.org/10.1101/2020.07.29.227959>

See Chapter 11’s references for Stamos, Sliwkowski & Eigenbrot (2002, PDB 1M17) and Mendez et al. (2019, ChEMBL, not directly used in this chapter but part of the same real receptor’s provenance chain). RDKit (Chapters 2, 4, 11) is reused for ligand bond-order assignment. PDBFixer and OpenMM-ML (both part of the OpenMM project, no separate primary paper of their own) are real, official companion tools, used via `pip install pdbfixer openmmml`.

All timings, RMSD/RMSF values, and feasibility-investigation numbers cited in Section 12.4 were computed directly by running `md_nnp_simulation.py` against the real bundled PDB 1M17 structure on 2026-08-21, not taken from a secondary source — see `results/md_nnp_results.json` to reproduce.